

Asymptotic

Regularity contracts
and structural work

VladimirReshetnikov/Asymptotic

Audited commit

651f2029d0b2cd4da9e4dfdf1f4275a124d23b99

10 September 2026

Assessment. The inspected code contains a derivative-contract propagation problem in the ordinary absolute-value observable path. A real, smooth input remainder can produce an absolute value with infinitely many cusps accumulating at the expansion point, while the result still advertises a classical derivative bound. Two recent repair paths also warrant targeted performance changes: repeated affine recognition during recursive interval evaluation, and repeated membership queries when finding discarded truncation blocks.

ID	New contribution	Evidence class
N01	Classical differentiability is not preserved by the pure-remainder Abs path.	Source + proof
N02	A linear affine recognizer is repeatedly invoked on nested nonlinear subtrees.	Source + exact counts
N03	Prefix truncation is processed as repeated general membership searches.	Source + cost model

What ran. Fourteen independent mathematical and structural tests and six patch-emitter fixture tests passed. They are not Wolfram package tests. Native Wolfram and Mathics execution did not succeed in this environment; the archive contains unexecuted native observations and conservative candidate edits for local validation.

Scope discipline. This article does not reissue the existing review backlog. It uses the consolidated four-wave register and intake records as its exclusion map, and directly compares the relevant portions of review 14. The ZIP contains this article, editable $\text{\LaTeX}$, models, recorded independent results, and reproduction tools. It is not an upstream package release.

Contents

1	Baseline, exclusions, and evidence	2
1.1	The reviewed version	2
1.2	How duplication was avoided	2
1.3	Evidence labels and execution limits	3
2	The invariant that matters: a jet is more than its finite part	3
3	N01: absolute value preserves a derivative contract it has not proved	4
3.1	The implementation path	4
3.2	A legitimate input family, not a forged object	4
3.3	Where the classical derivative disappears	5
3.4	The public reproduction candidate	5
3.5	A small conservative repair	6
3.6	The better implementation: track regularity inside the interpreter	6
3.7	Acceptance cases and an optional weak-derivative extension	7
4	N02: repeated affine probing makes a nonlinear fallback quadratic	7
4.1	A good local repair with a different global cost	7
4.2	A canonical, supported witness family	8
4.3	Executed independent counts	8
4.4	Repair with two passes, not more expansion	9
4.5	Caching scope and implementation tradeoffs	9
4.6	Native acceptance criteria	10
5	N03: finding a discarded suffix by repeated membership	10
5.1	Where the new work occurs	10
5.2	What is established, and what is not	10
5.3	A guarded prefix fast path	11
5.4	Why this is a useful low-risk patch	12
6	Implementation package and focused development sequence	12
6.1	What the supplied artifacts implement	12
6.2	Ordering the work	12
6.3	Tests should target the interaction, not only the repaired feature	13
7	Reproduction and recorded results	13
7.1	Independent checks that were executed	13
7.2	Native observations that remain to be run	14
7.3	Emitting and reviewing the candidate patch	14
8	Conclusion	15
A	Source coverage and exclusions	15
B	Artifact inventory	17

1 Baseline, exclusions, and evidence

1.1 The reviewed version

The audit is pinned to commit `651f2029d0b2cd4da9e4dfff1f4275a124d23b99`, whose timestamp is 10 September 2026, 13:42:02 UTC. The relevant library is `AsymptoticAnalysis`; its canonical implementation is under `src/Kernel1/`, while the root `AsymptoticAnalysis.wl` is a generated standalone. Findings refer to this immutable snapshot, not to whatever `main` contains when the article is read.[1, 2]

The immediately preceding commit, `e9142f7f5e3db241d5fffb8951c72606ed0c89aa9`, adds the rational-affine interval path discussed in N02. Its purpose is a valid repair of the earlier C20 problem. N02 concerns the cost of that newly installed path on *nonlinear* syntax; it does not allege that the original affine correctness/availability repair is still missing.[8, 11]

The source was read through pinned GitHub file requests. There was no successful complete local checkout. Source coverage was deliberately concentrated on the ordinary jet calculus, source transformations, exact interval evaluation, several special-function adapters, and selected Mathics compatibility modules. Appendix A distinguishes substantial file reads from selected windows. An inspected path without a retained new finding is not certified bug-free.

1.2 How duplication was avoided

The repository contains 36 imported review packages. Its register describes 231 attributed entries before consolidation of overlaps. Those are not 231 distinct open defects. The primary exclusion sources here are `CODE_REVIEW_STATUS.md`, the wave-3 and wave-4 intake records, and the imported-review index. Review 14’s affine and truncation sections were additionally read directly because they are the antecedents of N02 and N03.[3, 4, 5, 6, 7]

This is not a claim to have reread every page of all 36 articles. It is an explicitly documented crosscheck against the maintained consolidation, with closer original-source comparison where a candidate overlaps earlier work. The article does not repeat the existing recommendations about native-tail admission, generic derivative hypotheses, generic resource budgets, backward precision planning, or quantitative-bound transport as new findings.

New ID	Nearest existing entries	Distinct contribution retained here
N01	C13; D02; X04	A specific <code>Abs</code> operation carries an already legitimate derivative contract into a result for which that classical derivative does not exist on any punctured neighborhood. The witness is real; no missing initial derivative hypothesis is used.
N02	C20; review 14 N02	The installed repair repeatedly searches overlapping nonlinear subtrees. The new claim is a quadratic recognizer-call count for a stable Horner syntax family, not failure to preserve affine cancellation.
N03	C22; review 14 N05	The installed bound-transport routine uses repeated membership queries even when the retained rows are an identical prefix. The new proposal changes this selection algorithm, not the bound-transport theorem.

1.3 Evidence labels and execution limits

Three different kinds of statement occur below. A *source observation* identifies a concrete definition and control-flow path in the pinned code. A *mathematical or structural result* is proved independently of any kernel execution. A *native reproduction candidate* is executable Wolfram code whose actual outcome still needs to be recorded on the pinned package.

The independent Python tests exercise the counterexample identities, rational interval models, structural operation counts, prefix/fallback selection, and patch-emitter mechanics. They do not emulate Wolfram evaluation, package loading, option resolution, or Mathics. In particular, a passing independent test does not mean the public package call has been run. The native service returned connection errors, and neither a local Wolfram kernel nor an installable Mathics runtime was available. No native timing, patched native test, or full upstream suite result is claimed.

N01 is therefore a source-derived regularity-contract defect with a rigorous admissible witness, rather than an observed incorrect numerical coefficient. N02's repeated recursive recognizer work is explicit in the implementation. N03's comparison counts describe a conventional linear-membership model; they are not a statement about undocumented optimizations inside a particular Wolfram `MemberQ` implementation.

2 The invariant that matters: a jet is more than its finite part

The ordinary representation is usefully read as

$$F(y) = o(y) + p(y)(J(w(y)) + R(w(y))), \quad w \rightarrow 0^+, \quad (1)$$

where the finite jet has the form

$$J(w) = \sum_{\alpha < P} w^\alpha C_\alpha(\log w), \quad R(w) = \mathcal{O}(w^P(1 + |\log w|)^D).$$

The ordinary `SeriesRepresentation` stores an offset, a prefactor, a positive scale coordinate, a logarithmic variable, and a jet triple. Its `RemainderDerivativeOrder` is a separate contract, used by `seriesDerivative`; a magnitude estimate alone does not satisfy that guard.[10, 9]

For the examples in this article, $o = 0$, $p = 1$, and $w = y$, so no coordinate Jacobian or carrier regularity is involved. An order- k derivative contract means that the actual remainder has the relevant classical derivatives on an eventual punctured interval and that, for $0 \leq j \leq k$, they obey the matching bounds

$$R^{(j)}(y) = \mathcal{O}(y^{P-j}(1 + |\log y|)^D). \quad (2)$$

This interpretation is exactly what permits a routine to differentiate a stored remainder by lowering its power and its available derivative order. The implementation explicitly rejects a derivative request when the available order is too small.[10]

There are two independent closure questions. Does an operation preserve the *magnitude* estimate? Does it preserve *classical regularity and the matching derivative estimates*? Absolute value illustrates why the answers can differ. On real values it is Lipschitz:

$$||a| - |b|| \leq |a - b|.$$

Thus $|\mathcal{O}(y^P)| = \mathcal{O}(y^P)$ is a sound magnitude transport. It does not follow that absolute value preserves differentiability through zeros of its argument. No issue about complex absolute value or the interpretation of a fractional power is needed here.[14]

3 N01: absolute value preserves a derivative contract it has not proved

3.1 The implementation path

The ordinary helper `fwdAbs` starts with the following branch:

```
fwdAbs[j : {T_, P_, D_}, ell_, ass_] := Module[... ,
  If[T === {}, Return[j, Module]];
  ...]
```

For a pure remainder it returns exactly the same magnitude jet. That is the right magnitude estimate. For a nonempty jet, the rest of the routine attempts to establish the eventual sign from the leading logarithmic coefficient. [9]

The expression interpreter `seriesJetApply` delegates an `Abs` application to that helper. The public `SeriesObservable` then constructs the result using

```
j = seriesJetApply[body, x, d["Jet"], d, h, limit];
result = seriesMake[Join[d, <|"Jet" -> j|>],
  {"Observable", {s}, e, x}, h];
```

The replacement changes the jet but leaves the input's derivative-order field in `d`. The ordinary result constructor retains that field for an inexact jet. Consequently, the pure-remainder branch can preserve a positive derivative order without establishing smoothness of the absolute value. The relevant source locations are `SeriesOperations.wl`, the expression interpreter and lines 354–374, followed by the derivative guard at lines 536–558. [10]

3.2 A legitimate input family, not a forged object

Consider, for $0 < y < 1/4$,

$$R(y) = y^3 \sin(\log y), \quad H(y) = y + R(y). \quad (3)$$

Every derivative of R exists at every positive y . The elementary recurrence

$$\frac{d}{dy} [y^m (a \sin \log y + b \cos \log y)] = y^{m-1} [(ma - b) \sin \log y + (a + mb) \cos \log y] \quad (4)$$

shows that $R^{(j)}(y) = \mathcal{O}(y^{3-j})$ for every fixed j . In particular,

$$R'(y) = y^2 (3 \sin \log y + \cos \log y), \quad (5)$$

$$R''(y) = 5y (\sin \log y + \cos \log y). \quad (6)$$

A declaration of two matching derivatives for this input remainder is true.

Proposition 3.1. *The function H in (3) is the local inverse of a real smooth function F satisfying $F(x) = x + \mathcal{O}(x^3)$ on a positive one-sided neighborhood of zero.*

Proof. Since $|R'(y)| \leq \sqrt{10} y^2 < 4/16$, we have $H'(y) > 3/4$ on the stated interval. Also

$$\frac{15}{16} \leq \frac{H(y)}{y} \leq \frac{17}{16}.$$

Thus H is positive and strictly increasing, and has a smooth inverse F on its positive image. With $x = H(y)$,

$$|F(x) - x| = |y - H(y)| \leq y^3 \leq \left(\frac{16}{15}\right)^3 x^3.$$

This proves the required forward remainder estimate and local branch admissibility. $\square$

The point of this construction is that the input can be an ordinary inverse result with a declared omitted forward remainder. There is no need to manufacture an association, exploit an unvalidated saved object, assert false smoothness, or allow a complex unknown error. The family supplies an actual smooth inverse realizing the magnitude and derivative contracts used below. The argument uses the documented magnitude-plus-matching-derivative interpretation of those contracts, not an extra requirement of analyticity at the endpoint itself.

3.3 Where the classical derivative disappears

Set $g = R'$. Write

$$g(y) = \sqrt{10} y^2 \sin(\log y + \theta), \quad \theta = \arctan(1/3).$$

For every positive integer n ,

$$y_n = \exp(-\theta - n\pi) \tag{7}$$

is a zero of g , and $y_n \rightarrow 0^+$. These zeros are simple. Substitution into $g'(y) = 5y(\sin \log y + \cos \log y)$ gives

$$|g'(y_n)| = \sqrt{10} y_n > 0. \tag{8}$$

Proposition 3.2. *The magnitude estimate $|g(y)| = \mathcal{O}(y^2)$ is valid, but $|g|$ has no classical derivative at any y_n . Therefore it is not differentiable on any complete punctured neighborhood $(0, \delta)$ of zero.*

Proof. At a simple zero y_n , $g(y_n + h) = g'(y_n)h + o(h)$. Hence the left and right derivatives of $|g|$ are $-|g'(y_n)|$ and $|g'(y_n)|$, respectively. They differ by $2\sqrt{10}y_n$. Every punctured neighborhood contains such a point. $\square$

This is not a failure of a derivative *bound* at points where a derivative exists. Away from the zeros, the first derivative of $|g|$ is indeed $\mathcal{O}(y)$. It is a failure of the classical derivative's existence on the claimed eventual domain. Quietly interpreting the answer as an almost-everywhere or weak derivative would change the operation's meaning.

3.4 The public reproduction candidate

The following sequence uses the public APIs. The actual native results are not pre-populated in the archive.

```
s = AsymptoticInverse[x, {x, 0}, {y, 2},
  "InputRemainder" -> {3, 0}];
id = AsymptoticExpansion[y, {y, 0, 4}, "Backend" -> "Package"];
r = SeriesAdd[s, SeriesMultiply[id, -1]];
g = SeriesDifferentiate[r, 1, "RemainderDerivativeOrder" -> 2];
a = SeriesObservable[g, Abs[z], z];
SeriesDifferentiate[a]
```

The intended progression, derived from the inspected source, is:

Stage	Magnitude jet	Derivative order
r	$0 + \mathcal{O}(y^3)$	Initially undeclared
g	$0 + \mathcal{O}(y^2)$	1 after the truthful order-2 declaration
a	$0 + \mathcal{O}(y^2)$	Incorrectly retained as 1
Final differentiation	$0 + \mathcal{O}(y)$	Accepted on that retained premise

For the member H of the permitted inverse family, **a** represents $|g|$ from Proposition 3.2. The final classical differentiation must not be justified by the input contract alone. A corrected implementation should either refuse the last call, retain an explicitly weaker regularity status, or require additional information proving an eventual smooth branch.

The proof does not claim that the package executes the unspecified forward remainder as a numerical function. It reasons about the family denoted by the remainder contract. The existing numerical checker explicitly distinguishes its stored explicit equation from an unspecified input-remainder family; this audit preserves that distinction.[12]

3.5 A small conservative repair

A minimal correctness repair is to stop propagating a positive derivative order through an inexact ordinary observable containing **Abs**, unless the operation has separately proved a smooth local branch. The supplied patch emitter implements the conservative first step immediately after `seriesJetApply`:

```
If[! FreeQ[body, _Abs] && j[[2]] != Infinity,
  d = Join[d, <|"RemainderDerivativeOrder" -> 0|>];
```

This is intentionally broader than the bad pure-remainder case. It can lose a useful derivative guarantee when the absolute-value argument actually has a proved eventual nonzero sign. It does not change the finite jet or its magnitude bound. Exact output jets still receive exact-result treatment. The patch is a correctness-first candidate, not a claim of a maximal regularity implementation.

Keeping this change in the ordinary `SeriesObservable` path also means that replaying an observable recipe through that patched path applies the same conservative policy. No external wrapper should attach a one-time warning while leaving a replay route capable of silently restoring the old guarantee.

3.6 The better implementation: track regularity inside the interpreter

The more precise repair is to make the expression interpreter return both its jet and an available smoothness order. For an absolute-value node, the transfer rule is local and small:

$$k_{\text{out}} = \begin{cases} k_{\text{in}}, & \text{if the complete argument is eventually strictly positive or negative,} \\ \infty, & \text{if the complete argument is exactly zero,} \\ 0, & \text{otherwise, using only the real Lipschitz estimate.} \end{cases} \quad (9)$$

A nonempty leading block with a proved sign can provide the first case because the stored error is asymptotically smaller than that block. An empty finite jet cannot provide it merely by carrying a large derivative-order number.

The interpreter should aggregate this order through nested expressions, not inspect only the outermost head. A magnitude-neutral intermediate **Abs** can invalidate regularity before a later

arithmetic operation. Conversely, algebraic identities such as $|g|^2 = g^2$ for real g may restore smoothness, but that is a separate proved identity, not a reason to retain the old order by default. This narrowly scoped regularity transfer is the development opportunity made concrete by N01.

3.7 Acceptance cases and an optional weak-derivative extension

The regression must inspect both the outer association and `SeriesRepresentation`; a downgrade in only one place can be bypassed by `seriesData`. The primary case is the public chain above. Controls should include exact zero, an exact sign-known polynomial, a nonempty sign-known jet with an inexact tail, nested absolute values, and replay of the observable recipe. A sign-aware implementation should retain the useful order in the proved-sign control; the conservative patch intentionally need not do so.

An optional future interface could distinguish classical derivatives from almost-everywhere derivatives of real Lipschitz observables. That would require an explicit change of contract. For this witness the first weak derivative is locally bounded, but the distributional second derivative of $|g|$ also contains point masses with weights $2|g'(y_n)|$. An ordinary power-log magnitude remainder does not represent that discrete distribution. A weak-derivative mode therefore must not be implemented by simply turning off the classical guard.

4 N02: repeated affine probing makes a nonlinear fallback quadratic

4.1 A good local repair with a different global cost

The new `certAffinePair` recognizes rational affine syntax by carrying a pair (a, b) for $ax + b$. Sums add their pairs. Products combine pairs while refusing a product of two nonconstant affine factors. Other heads fail the recognition attempt. One invocation visits each node of its admitted arithmetic tree at most once, apart from exact rational arithmetic. The source comment's linear-tree description is reasonable for that *single invocation*. [11]

The important integration point is different. Before the ordinary `Plus` or `Times` interval fallback, `certEnclose` invokes `certAffineRange`, which invokes `certAffinePair`. If that test fails, `certEnclose` recursively processes the children, each of which may trigger another full affine-recognition attempt. Successful affine roots are cheap; deeply nested nonlinear roots repeatedly expose overlapping subtrees.

```
MemberQ[{Plus, Times}, Head[expression]] &&
  (affine = certAffineRange[expression, x, interval]) != $Failed,
certRoundInterval[affine, ctx],
Head[expression] === Plus,
  Fold[certAdd[#1, #2, ctx] &, {0, 0},
    certEnclose[#, x, interval, ctx] & /@ (List @@ expression)],
...
```

This integration is visible at `InverseCertificates.wl`, lines 103–148. The finding concerns recognizer work before the old conservative fallback; it does not question the soundness of an interval returned by that fallback. [11]

4.2 A canonical, supported witness family

Use the unevaluated-by-expansion Horner family

$$h_0(x) = 1 + x^2, \quad h_{d+1}(x) = 1 + xh_d(x). \quad (10)$$

In Wolfram syntax this is

```
expr = Nest[1 + x # &, 1 + x^2, depth];
```

The seed $1 + x^2$ is deliberate. Starting from x^2 alone allows the first multiplication to simplify to x^3 , changing the base-case count. The chosen seed leaves alternating **Plus** and **Times** nodes under ordinary canonical evaluation without calling **Expand**. Every h_d is a supported polynomial, and the interval $[0, 1/2]$ avoids any domain difficulty. There is no unsupported special function at the bottom of the expression.

Let A_d be the number of **certAffinePair** invocations in one recognition attempt on h_d . The **Power** node x^2 is rejected immediately by the affine recognizer, so

$$A_0 = 3, \quad A_d = A_{d-1} + 4 = 4d + 3.$$

For $d \geq 1$, the intermediate product xh_{d-1} costs $1 + 1 + A_{d-1} = 4d + 1$ recognizer invocations. The recognizer eagerly maps over the children before it tests whether a failure occurred, so the failed nonlinear subtree is traversed again at the next recursive interval level.

Proposition 4.1. *In the inspected recursive control structure, enclosing h_d causes*

$$C_d = 4(d + 1)^2 - 1 \quad (11)$$

invocations of the affine-pair recognizer. Thus recognizer work is quadratic in d , even though the input syntax has linear size.

Proof. At depth zero, the only affine attempt is on $1 + x^2$, so $C_0 = 3$. For $d \geq 1$, the evaluator first attempts the whole sum, then the product, then recursively encloses h_{d-1} :

$$C_d = C_{d-1} + (4d + 3) + (4d + 1) = C_{d-1} + 8d + 4.$$

Summing yields $C_d = 3 + 4d^2 + 8d = 4(d + 1)^2 - 1$. □

This is a count of explicit recursive calls, not a claim that rational arithmetic has constant bit cost. Integer/rational operand sizes, expression hashing, list allocation, and interval arithmetic can add work. They do not eliminate this repeated traversal in the source algorithm. The claim is also narrower than a timing prediction for a complete certificate: a certificate evaluates several expressions, including a derivative, and its surrounding proof work may dominate a small example.

4.3 Executed independent counts

The supplied model constructs an explicit syntax tree with **Plus**, **Times**, an integer-power node, rational constants, and a variable. It implements the inspected recognition/fallback control flow and compares it with a single annotation pass. These are *model operation counts*, not native performance measurements.

Horner depth d	Repeated recognizer calls C_d	Annotation nodes $4d + 5$
4	99	21
8	323	37
16	1,155	69
32	4,355	133
64	16,899	261
128	66,563	517

The annotation-node count uses the model’s explicit literal exponent node; it is not Wolfram’s **LeafCount**. All tested depths matched the closed-form recognizer count. The two evaluators also agreed on 300 randomized rational arithmetic trees. Their interval outputs enclosed 2,100 exact rational sample values. Sampling is a useful regression check, while interval soundness rests on the elementary operation rules and their inductive composition.

4.4 Repair with two passes, not more expansion

A direct remedy is to annotate the expression tree once, then evaluate intervals using the annotation. For each node retain either a rational affine pair or **NotAffine**. All children are annotated bottom-up. Importantly, an unsupported affine head such as **Power**, **Log**, or **Exp** does not prevent useful affine annotations within its children.

The interval evaluator then behaves as follows. At a node carrying a pair, it computes the affine endpoint range exactly and applies the existing outward rounding at the same semantic point as today. At a node without a pair, it uses the existing interval rule and the already annotated children. No polynomial expansion, solving, or approximate coefficient test is required.

Lemma 4.2. *For the rational arithmetic fragment implemented in the independent model, replacing repeated recognition by bottom-up affine annotations preserves the result of the original recognition-plus-interval algorithm.*

Proof. Induct on the expression tree. The annotation computes exactly the same pair recurrence as an independent recognition call; therefore both evaluators take the affine shortcut at the same nodes and use the same exact endpoint formula. At every other supported node, they apply the same interval operation to child results that agree by induction. $\square$

The production version should retain the existing rational/dyadic interval primitives. The model deliberately does not reimplement the complete exponential and logarithmic enclosure engine. Its purpose is to establish the structural transformation and its local equivalence obligations, not to replace a trusted certificate evaluator with an untested partial implementation.

A single pass returning both a pair and an interval is possible, but a careless implementation computes rounded child intervals or raises child-domain failures before discovering that the parent is exactly affine. The two-pass design cleanly separates structural recognition from semantic interval evaluation and avoids that ordering mistake. Annotation must remain arithmetic metadata, not a proof that a nonaffine expression is continuous or in domain.

4.5 Caching scope and implementation tradeoffs

A cache can also remove repeated recognizer calls, but a cache keyed by complete expression trees may still pay repeated structural hashing or equality costs. Its actual cost depends on sharing and kernel representation. An explicit annotated tree gives a straightforward linear structural-pass argument without requiring such a representation assumption.

Affine annotations depend on the expression and chosen source symbol, but not on the verification interval or enclosure precision. Interval values depend on all of those. Mixing the two cache lifetimes would be incorrect: adaptive refinement changes the interval and arithmetic context. A suitable implementation can reuse a structural plan through several interval evaluations while recomputing all interval-dependent values.

Annotation increases live structural storage by $\mathcal{O}(N)$ for an N -node tree. For tiny expressions, the existing local probe may be cheaper. A hybrid can keep the simple path below a small structural threshold and use the annotated path for larger trees, but the threshold should be determined by measurements rather than guessed in this audit. The correctness invariant is the same in either case: exact affine arithmetic remains before rounding.

4.6 Native acceptance criteria

The native runner supplies the stable Horner family at depths 8, 16, 32, and 64, records actual `LeafCount`, and times `certEnclose` separately from construction. It intentionally does not advertise an unmeasured speedup. A production benchmark should additionally count recognizer invocations, with instrumentation that distinguishes entry from exit tracing and does not itself perform quadratic work.

Tests should compare old and planned enclosures on exactly affine, nested nonlinear, mixed affine/transcendental, and unsupported expressions. Check the same failures and interval inclusions, not only elapsed time. In particular, retaining the affine repair's existing success cases is a regression obligation, not a new finding in this article.

5 N03: finding a discarded suffix by repeated membership

5.1 Where the new work occurs

The recent `seriesTransportRemainderBound` routine obtains the source rows and newly retained rows, then computes the discarded set as follows:

```
before = d["Jet"][[1]];
after = r["SeriesRepresentation"]["Jet"][[1]];
removed = Select[before, ! MemberQ[after, #] &];
If[! SubsetQ[before, after], Return[result, Module]];
```

These are lines 516–518 of `SeriesOperations.wl`. The routine subsequently uses the known discarded expression to transport an existing quantitative bound. It also handles the case in which no retained block was removed. [10]

The bound-transport theorem is already an earlier-review contribution and is not being proposed again. The new issue is the selection algorithm used by the implemented repair. Ordinary exponent truncation of a canonical ordered jet normally retains an identical prefix. That common case does not require a general membership query for every source row.

5.2 What is established, and what is not

Let the input have N distinct rows and the result retain the first M rows. The source expression performs N separate `MemberQ` calls on a list of length M , followed by a subset check. `MemberQ` is a pattern-membership operation; the official documentation does not promise an asymptotic complexity or expose a reusable indexed set for these calls.[15]

Under a straightforward linear scan, the row comparisons in the first statement alone are

$$\sum_{i=1}^M i + (N - M)M = NM - \frac{M(M - 1)}{2}. \quad (12)$$

For a no-op truncation, $M = N$, this is $N(N + 1)/2$. This is the exact count in the supplied linear-search model, not an instrumented count of the Wolfram kernel’s internal pattern matcher. An undocumented native optimization could change the actual timing or comparison behavior. The source still issues repeated general queries where a proved prefix relation is sufficient.

Blocks, no-op case	Linear-membership model comparisons	Prefix row comparisons
16	136	16
64	2,080	64
256	32,896	256
512	131,328	512

The table excludes the old subset check and the cost of comparing large coefficient expressions. It therefore should not be read as a native wall-clock speedup estimate. It identifies a local avoidable pattern and supplies a small alternative whose structural work is explicit.

5.3 A guarded prefix fast path

The candidate replacement is:

```
If[Length[after] <= Length[before] &&
  Take[before, Length[after]] === after,
  removed = Drop[before, Length[after]],
  removed = Select[before, ! MemberQ[after, #] &];
If[! SubsetQ[before, after], Return[result, Module]]];
```

The length guard is important: it prevents an out-of-range **Take** from becoming the new diagnostic behavior. The structural equality check is also important. The patch does not merely assume that every stored representation uses the same canonical coefficient syntax. If normalization has changed a row, or the result is not a prefix, it falls back to the old selection and subset policy.

Proposition 5.1. *For a well-formed canonical jet with distinct rows, suppose*

$$after = \text{Take}(before, M).$$

The proposed fast path returns the same discarded rows as the original membership expression, in the same order, and the original subset test would succeed.

Proof. The first M rows occur in **after**; every remaining distinct row does not. The original **Select** therefore retains exactly the suffix beginning at $M + 1$. That is **Drop[before, M]**. Every row in the prefix belongs to the source, so the subset relation follows without a second membership pass. $\square$

This reasoning is deliberately restricted to canonical jets with distinct rows. It is not a new validation rule for arbitrary user-built associations. The fallback remains for other structurally admissible situations. It preserves the original routine’s handling rather than changing the meaning of membership or attempting to use a symbolic inequality between coefficient expressions.

5.4 Why this is a useful low-risk patch

The fast path affects only the extraction of the known discarded finite part. The subsequent bound formula, conditions, signed-bound policy, contract record, and presentation of integer Dirichlet powers remain untouched. This keeps the change separable from mathematical bound improvements already in the backlog.

A semantic no-op is a particularly useful benchmark because it isolates a case where no discarded contribution is needed. The candidate still compares a prefix, but avoids re-searching it for each row. For a genuine truncation it extracts the suffix directly. Its cost is linear in the structural size of the rows inspected or copied; coefficient-expression bit/term complexity is not assumed constant.

The independent suite checked 500 randomly generated prefix cases, reordered and nonprefix fallbacks, empty lists, complete retention, and refusal of an invalid subset in the model. The native runner additionally specifies no-op truncations of public Zeta expansions. Those native observations have not been executed here. Their actual constructor and truncation costs may include other work, so the helper and the whole operation should be profiled separately.

6 Implementation package and focused development sequence

6.1 What the supplied artifacts implement

The archive contains a standard-library Python model and an executable test runner. The affine model includes both repeated recognition and a bottom-up annotation plan. The truncation model includes the repeated linear-membership algorithm and the guarded prefix alternative. The regularity witness is checked through an exact derivative-coefficient recurrence, elementary rational bounds, and supplementary numerical one-sided difference quotients.

The patch emitter produces two candidate edits to the canonical `src/Kernel/SeriesOperations.wl`: the conservative `Abs` regularity downgrade, and the guarded prefix fast path. It does not modify the repository. It verifies the audited Git commit by default and requires unique exact source anchors. An explicit override permits an unversioned source tree while retaining anchor validation. Its fixture tests cover both edits independently, missing and duplicate anchors, already changed input, and preservation of unrelated text.

No production `InverseCertificates.wl` patch is supplied. N02 has an implemented independent reference design and a native benchmark specification, but integrating the annotation plan into all certificate expression rules and failure paths deserves a full native validation pass. Supplying a partial certificate-engine replacement as if it were complete would be misleading.

6.2 Ordering the work

Step	Change	Completion criterion
1	Apply a conservative regularity downgrade for inexact absolute-value observables.	The N01 chain no longer obtains a classical derivative from the transported input contract; exact controls remain correct.
2	Refine that repair into sign-aware regularity transfer inside the expression interpreter.	Proved-sign cases preserve their legitimate order; unknown zero-crossing cases do not; nested observables and recipe replay agree.

Step	Change	Completion criterion
3	Install the guarded prefix selection path.	Prefix/no-op cases select the same discarded rows and preserve the existing quantitative-bound semantics; nonprefix cases retain the fallback.
4	Introduce a certificate structural annotation plan.	Affine decisions and outward interval behavior match the old evaluator while Horner-family recognition work becomes linear in tree size.
5	Add narrow scaling and semantic-composition regressions.	Structural work counts and regularity transitions are tested separately from numerical values and total timings.

This sequence is intentionally local. It does not reproduce the already extensive roadmap for additional scales, special functions, native-backend coverage, integration, general proof objects, or multivariate systems. The opportunities identified here are operation-specific regularity transfer, reusable interval-independent affine plans, and exploiting the exact prefix invariant of truncation.

6.3 Tests should target the interaction, not only the repaired feature

N01 is invisible to tests that inspect only `Normal[result]` and its remainder power. The invalid fact is in the derivative capability consumed by a later operation. The essential regression is therefore a sequence: construction with an honest remainder family, arithmetic cancellation, differentiation with a legitimate declaration, absolute value, and a second classical differentiation.

N02 is invisible to tests containing only successfully affine expressions. A recognizer that is fast when it succeeds can be expensive when it repeatedly fails just above an old nonlinear subtree. The depth family deliberately holds the mathematical fragment fixed and changes only nesting.

N03 is invisible to tests of bound values alone. The implemented bound can be perfectly valid while its no-op transport makes unnecessary general searches. The acceptance suite should retain both the semantic checks and a narrowly scoped structural-work observation. A total-time threshold alone is too noisy to explain which phase regressed.

These three test shapes are distinct from a blanket request for more tests. They are specific interaction witnesses and size families derived from the current implementation.

7 Reproduction and recorded results

7.1 Independent checks that were executed

Run from the archive directory:

```
python code/verify_independent.py
python code/test_patch_emitter.py
```

Python 3.10 or later and its standard library are sufficient. The first command writes `evidence/independent_results.json` and its text transcript. The second writes the patch-fixture result record and transcript. The delivered records come from actual executions during this audit.

Suite	What was checked	Result
Independent model tests	Derivative recurrence, cusp witnesses, inverse-family bounds, affine recognition, exact interval equivalence, prefix/fallback selection, and structural count formulas.	14/14
Patch-emitter fixtures	Unique-anchor transformations, individual edits, missing/duplicate anchors, and refusal to reapply edits.	6/6

The independent suite includes 300 randomized expression trees, 2,100 exact rational evaluation points inside their intervals, and 500 randomized prefix cases. These are subcases within the 14 tests, not thousands of package test passes. Random seeds are fixed in the source. Mathematical proofs in the article supply the general statements; the finite samples are supplementary executable checks.

The cusp samples numerically approximate left and right difference quotients normalized by y_n . They approach $-\sqrt{10}$ and $+\sqrt{10}$. This numerical calculation is not used to infer non-differentiability: the exact local expansion in Proposition 3.2 already proves it.

7.2 Native observations that remain to be run

Use a fresh kernel and an existing checkout of the pinned revision. For example, in PowerShell:

```
$env:ASYMPTOTIC_REPO = 'C:\src\Asymptotic'
wolframscript -file code/run_native.wls
```

For a POSIX shell:

```
ASYMPTOTIC_REPO=/path/to/Asymptotic \
wolframscript -file code/run_native.wls
```

The script reads the canonical modular loader. It records the kernel version, the observed Git commit when available, the N01 public chain, an exact sign-known control, Horner-family interval timings, and Zeta no-op truncation observations. Individual observations have a bounded execution time. It does not download code, install dependencies, edit the repository, or claim to run the upstream complete suite.

The script's exit status distinguishes a runner/load/output failure from successfully writing observations. A zero exit code is *not* a correctness verdict: an observation can show the baseline defect, a timeout, a message, or a rejected operation. Inspect the serialized results. The delivered archive does not contain a fabricated `native_observations.json`; that file is created only by an actual native execution.

The runner is a Wolfram-kernel observation tool, not a promise of compatibility with every Mathics release. The mathematical counterexample and the structural algorithms do not depend on a particular kernel, but public routing, native **Series**, pattern matching, and execution timing do. A Mathics validation should record its own results separately rather than relabeling Wolfram or Python results.

7.3 Emitting and reviewing the candidate patch

```
python patches/emit_candidate_patch.py /path/to/Asymptotic \
> candidate.diff
```

Use `-which regularity` or `-which prefix` to emit one change only. The emitter is read-only.

It refuses an unexpected Git commit by default and refuses missing or duplicate anchors. The explicit `-skip-commit-check` option is for a consciously selected unversioned or modified source tree; it does not disable the anchor checks.

The source anchors were compared with the pinned reads. The transformation was fixture-tested, but it was not applied to a complete local upstream checkout in this audit. The generated patch still needs review, native execution of the focused cases, and the repository’s normal validation. Rebuild the standalone using `validation/build_standalone.py`; editing the generated distribution alone would not update the canonical source.[2]

The regularity patch is intentionally conservative and can reduce supported differentiation in some valid sign-known inexact cases. The prefix patch has a narrower semantic footprint and retains the old fallback. Neither is presented as a completed production release.

8 Conclusion

The strongest new result is N01. The existing derivative guard can be obeyed honestly at the input and still be bypassed indirectly by an operation that preserves magnitude but not differentiability. The example is not based on a complex branch, a malformed object, or a missing initial derivative estimate. Its infinitely many simple zeros explain exactly which premise is lost.

N02 and N03 concern the implementation of two useful recent fixes rather than a rejection of those fixes. Affine cancellation should remain exact before rounding, but the recognition plan need not be rediscovered at every nonlinear subtree. Quantitative truncation bounds should remain preserved, but a known prefix can be removed without repeated general membership searches.

The immediate actionable changes are small: downgrade unproved regularity in the absolute-value observable path and add the guarded prefix fast path. The larger follow-on is a reusable, interval-independent affine annotation plan. Together they give specific correctness and performance improvements without repeating the repository’s established development backlog.

A Source coverage and exclusions

The table is a source-reading record, not a test-coverage percentage. “Selected” means substantial relevant windows rather than a verified complete-file read. Several large connector responses were windowed or truncated, so completeness is not inferred from a request for an entire file.

Source under <code>src/Kernel/</code>	Read scope	Focus and disposition
<code>AsymptoticAnalysis.wl</code>	Selected	Public inverse assembly, precision-tracked jets, arithmetic, <code>fwdAbs</code> , representation, formatting, and residual dispatch. N01 uses the absolute-value branch.
<code>SeriesOperations.wl</code>	Selected	Representation normalization, observables, composition, truncation, derivative guard, and refinement entry paths. N01 and N03 retained.

Source under <code>src/Kernel/</code>	Read scope	Focus and disposition
<code>InverseCertificates.wl</code>	Selected	Rational interval primitives, affine recognition, recursive enclosure, source conditions, seed and adaptive certificate flow. N02 retained.
<code>SeriesEnvelopeArithmetic.wl</code>	Substantial	Composite-envelope arithmetic and approach handling. No additional finding retained.
<code>SourceCoordinates.wl</code>	Selected	Logarithmic/exponential source charts, observable restoration, and error transport. No additional finding retained.
<code>GammaForward.wl</code>	Substantial	Positive-factor logarithmic normalization and forward carrier extraction. No additional finding retained.
<code>GammaInverse.wl</code>	Substantial	Ordered coefficient recurrence, affine parameters, omitted block bound, and truncation. No additional finding retained.
<code>SpecialFunctionAdapters.wl</code>	Selected	Erfc, Gamma/LogGamma, and local threshold model construction. No additional finding retained.
<code>ParameterizedSpecialFunctions.wl</code>	Substantial	Fixed-parameter analytic and Frobenius adapters. No additional finding retained.
<code>DirichletSpecialFunctions.wl</code>	Substantial	Zeta/Lerch block construction and explicit bound metadata; supports the N03 benchmark specification.
<code>NumericalInverseChecks.wl</code>	Substantial	Stored-equation scope and numerical source-domain checks. No additional finding retained.
<code>ExactTermination.wl</code>	Substantial	Bounded symbolic verification of inverse termination. No additional finding retained.
<code>NativeCompatibility.wl</code>	Selected	Native-result distinction and backend/held request entry; checked the reproduction option spelling. No additional finding retained.
<code>MathicsAlgebra.wl</code>	Substantial	Algebraic substitutes and list helpers; previously catalogued concerns excluded.
<code>MathicsAssumptions.wl</code>	Substantial	Realness/sign proof helpers and assumption walking. No additional finding retained.
<code>MathicsCalculus.wl</code>	Substantial	Perturbative inverse expression and logarithmic Euler derivative. No additional finding retained.
<code>MathicsSimplification.wl</code>	Substantial	Simplification, assumptions, and Series/Limit wrappers. Previously catalogued concerns excluded.

Source under <code>src/Kernel/</code>	Read scope	Focus and disposition
<code>MathicsTaylor.wl</code>	Substantial	Restricted defining Taylor series. Previously catalogued parameter restrictions excluded.
<code>MathicsSpecialFunctions.wl</code>	Substantial	Finite Stirling forward adapter. No additional finding retained.

The ordinary calculus tests in `src/Tests/SeriesOperations.wlt` were also read, notably their explicit derivative-contract tests and exact finite-tail control. They were not executed. Architecture descriptions of other modules were read in the kernel README, but that is not counted as inspection of those modules' implementation.[13, 2]

The maintained review register was used to exclude already catalogued work even when a related code path remained interesting. Consequently, the absence of a new finding in the table should not be interpreted as a claim that every existing concern is resolved. That would contradict the purpose of a nonduplicating incremental audit.

B Artifact inventory

File	Role
<code>article.tex</code> , <code>article.pdf</code> <code>README.md</code>	Editable article and rendered version. Reproduction commands and execution boundaries.
<code>code/audit_models.py</code>	Independent rational/structural reference models and derivative recurrence.
<code>code/verify_independent.py</code>	Executed 14-test suite; writes its actual result records.
<code>code/test_patch_emitter.py</code> <code>code/run_native.wls</code>	Executed six-test synthetic fixture suite. Unexecuted native observations; records actual results when run locally.
<code>code/check_wl_lexical.py</code>	Delimiter, string, and nested-comment balance check; not a native parser.
<code>patches/emit_candidate_patch.py</code>	Read-only emitter for the two conservative candidate edits.
<code>evidence/independent_results.json</code>	Actual independent counts, subcase totals, and cusp samples.
<code>evidence/independent_tests.txt</code>	Actual independent-test transcript.
<code>evidence/patch_fixture_results.json</code>	Actual patch-fixture outcome.
<code>evidence/patch_fixture_tests.txt</code>	Actual fixture-test transcript.
<code>evidence/audit_manifest.json</code>	Snapshot, scope, finding status, and execution record.
<code>evidence/novelty_crosswalk.csv</code>	Concise mapping to nearest previous entries without reissuing them.
<code>evidence/source_inventory.csv</code>	Pinned source-reading inventory.
<code>evidence/wl_lexical_check.json</code>	Actual lexical-balance check outcomes.
<code>build.sh</code>	PDF build command sequence.

No upstream package distribution, font files, checksum files, or fabricated native result files are included.

References

- [1] Vladimir Reshetnikov. *Asymptotic*, audited commit 651f2029d0b2cd4da9e4dfdf1f4275a124d23b99. <https://github.com/VladimirReshetnikov/Asymptotic/tree/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99>. Repository README and commit metadata, accessed 10 September 2026.
- [2] *Canonical kernel layout*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/README.md>.
- [3] *Imported code-review index*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/README.md>.
- [4] *Code review implementation status*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/CODE_REVIEW_STATUS.md.
- [5] *Wave 3 intake*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/WAVE_3_INTAKE.md.
- [6] *Wave 4 intake*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/WAVE_4_INTAKE.md.
- [7] *Incremental technical audit*, imported review 14. Affine and truncation sections, especially source lines 263–355 and 470–580. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-2/code-review-14/article.tex>.
- [8] *Evaluate rational affine subexpressions exactly in certificates (C20)*, commit of 10 September 2026, 13:37:49 UTC. <https://github.com/VladimirReshetnikov/Asymptotic/commit/e9142f7f5e3db241d5ffb8951c72606ed0c89aa9>.
- [9] *AsymptoticAnalysis.wl*, canonical main module. Relevant definitions: `fwdAbs`, precision-tracked jets, inverse assembly, and `GeneralizedSeries` accessors. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/AsymptoticAnalysis.wl>.
- [10] *SeriesOperations.wl*. Relevant source windows: 305–374 (observable interpretation and construction), 506–558 (bound transport and differentiation), and the representation helpers near the beginning of the module. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/SeriesOperations.wl>.
- [11] *InverseCertificates.wl*. Relevant source windows: 95–151 (affine recognizer and enclosure dispatch), 251–327 (certificate attempt), and 357–484 (public certificate orchestration). <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/InverseCertificates.wl>.
- [12] *NumericalInverseChecks.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/NumericalInverseChecks.wl>.
- [13] *SeriesOperations.wlt*, upstream ordinary calculus tests. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Tests/SeriesOperations.wlt>.
- [14] Wolfram Research. *Abs*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Abs.html>. Accessed 10 September 2026. Only the real absolute-value meaning is used here.
- [15] Wolfram Research. *MemberQ*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/MemberQ.html>. Accessed 10 September 2026. Documents pattern-membership semantics, not a complexity guarantee for the audit’s repeated-list-query pattern.